

Audio Speech-to-Language Classification by Region

Vincent Wang
vincew8@uci.edu
University of California, Irvine
Irvine, California, USA

Keerthana Girish
keerthag@uci.edu
University of California, Irvine
Irvine, California, USA

Medha Bhattacharya
mbhatta1@uci.edu
University of California, Irvine
Irvine, California, USA

Melina Pouya
mpouya@uci.edu
University of California, Irvine
Irvine, California, USA

Eric Cao
caoe5@uci.edu
University of California, Irvine
Irvine, California, USA

Abstract

This project studies audio-based language classification using neural network models in regional subsets of the Google FLEURS dataset. Our team compares multiple architectures, including feed-forward neural networks, convolutional neural networks, recurrent models, and transformer-based models, to understand how well different architectures capture acoustic and language-specific patterns in speech. The report presents the organization of the data set, the pre-processing pipeline, the architecture design, the training setup, and the experimental results across multiple regional language groups.

CCS Concepts

• **Computing methodologies** → **Neural networks**; *Speech recognition*; *Supervised learning by classification*.

Keywords

audio language classification, Google FLEURS, neural networks, speech processing, deep learning

1 Introduction

This project investigates audio-based language classification using neural network models on regional subsets of the Google FLEURS dataset. Unlike text-based language identification, which utilizes lexical information and is generally less noisy, speech-based classification must rely directly on acoustic and phonetic cues in the waveform. This introduces an additional complexity, where the model must learn nuances in pronunciation, tone, and pitch for better predictions.

This task becomes particularly challenging when limiting the classification scope to languages from the same geographic region. Languages that are closely related often share similar phonetic patterns and historical linguistic roots, resulting in highly overlapping acoustic features. This overlap reduces inter-class separability and increases the likelihood of misclassification. For example, while it is relatively easy to distinguish between languages such as Spanish and Chinese, it can be much more difficult to identify the subtle differences in pronunciation and speech patterns between languages like Polish and Slovak.

Our team compares multiple model families for speech-to-language classification, including feed-forward neural networks (FNNs), convolutional neural networks (CNNs), long short-term memory networks (LSTMs), and transformer-based models. The goal of the

project is to understand how different architectures perform on the same classification task and how architectural design affects generalization, confusion patterns, and overall accuracy across regional language groups.

2 Data

We use the Google FLEURS dataset [4], a multilingual speech dataset containing audio utterances and language labels across many languages. For this project, the data was divided into regional subsets so that each experiment could focus on a region-specific multi-class language classification problem.

2.1 Dataset Overview

FLEURS is a multilingual read-speech benchmark built from the FLoRes sentence collection and covers 102 languages. Each example contains a speech waveform together with its language label, normalized and raw transcription, and additional metadata such as speaker gender and language-group identifiers. The dataset is especially suitable for multilingual speech classification because the utterances are collected in a consistent recording setting across languages, and the language-identification task is defined over n -way parallel sentences, which reduces domain mismatch across classes and makes comparisons across languages more controlled.

The dataset provides predefined train, validation, and test splits for each language [4]. In our project, these language-specific splits were merged within each regional subset to form region-level multi-class classification tasks. FLEURS provides roughly 10 hours of training audio per language, and the exact number of utterances varies by language. After regional merging, the resulting datasets used in our experiments ranged from several thousand to tens of thousands of examples. For example, the merged regional splits included Western European (23,150/900/1,150), Eastern European (23,856/4,640/10,528), South Asian (15,134/3,346/4,186), CJK (7,756/904/1,528), South-East Asian (15,075/1,719/3,645), Central Asia + MENA (17,892/2,604/3,588), and Sub-Saharan African (15,480/285/615) utterances for train/validation/test, respectively.

An important characteristic of FLEURS is that the train, validation, and test splits are partitioned by speaker identity. Each speaker appears in exactly one split and never across splits. This means that a model evaluated on the validation or test set will always encounter voices it has never heard during training. While this design reflects a realistic deployment scenario, it poses a significant challenge for models that lack speaker-invariant representations. A model that

learns to associate a particular language with the acoustic characteristics of specific training speakers, such as their pitch, speaking rate, or vocal tract resonances, will generalize poorly to unseen speakers in the validation and test sets. This speaker disjointness is therefore an important factor when interpreting the gap between training and validation accuracy observed in our experiments.

2.2 Regional Language Grouping

To make the language-identification problem more structured and interpretable, we divided the dataset into regional groups. The experiments in this project used seven groups: Western European (25 languages), Eastern European (16 languages), South Asian (14 languages), Chinese + Japanese + Korean (4 languages), South-East Asian (9 languages), Central Asia + Middle East + North Africa (12 languages), and Sub-Saharan African (15 languages). These groups were based on the broad geographical organization of FLEURS, with some project-specific curation in regions where a smaller working subset was used for training.

Grouping languages by region was useful for two reasons. First, it reduced the size of the classification problem from a single 102-way task to several smaller and more interpretable multi-class tasks. Training a single model on all 102 languages simultaneously would have been impractical at our scale, as the full FLEURS dataset spans over 1,000 hours of audio and a 102-way classification head requires significantly more model capacity and compute power than our available GPU budget allowed. Second, it allowed us to study confusion patterns among languages that are geographically and often acoustically or phonologically related. This makes the experiments more informative than a single global classification setting, since performance can be interpreted in terms of region-specific difficulty, similarity between languages, and how well a model separates closely related language families.

2.3 Preprocessing Overview

All experiments in this study use the same preprocessed speech representation. Each utterance is converted into a variable-length sequence of 80-dimensional acoustic feature vectors and stored as a PyTorch sample containing the feature matrix $\mathbf{x}$, the class label y , and the language name. As a result, every example is represented as a tensor of shape $T \times 80$, where T varies with utterance duration while the feature dimension remains fixed. These preprocessed samples are saved as .pt files and organized into train, validation, and test splits for each regional language subset.

To ensure consistent comparison across models, the preprocessed data are grouped into balanced regional subsets and loaded directly during training. This design keeps the input representation identical across architectures, so differences in performance can be attributed to the models themselves rather than to changes in preprocessing. Because the utterances are stored as frame-level feature sequences, the preprocessing stage preserves temporal acoustic structure while also providing a standardized representation that can be reused across multiple model families.

3 Methods: Architecture Design and Training

Our project compares several neural architectures for audio speech-to-language classification. All models use the same underlying

dataset, but differ in how they represent, aggregate, and classify acoustic information. This section first presents the shared processing and training setup, then provides architecture-specific subsections for each model family explored by the team.

3.1 Shared Processing Pipeline

All architectures in the project begin with a common preprocessing stage that converts each raw waveform into a feature representation. These preprocessed features are then passed into model-specific architectures for training and evaluation.

The shared workflow is as follows. First, raw speech utterances from FLEURS are assigned to one of the regional subsets used in the project. Each utterance is then converted into an 80-dimensional frame-level acoustic representation, producing a variable-length feature sequence of shape $T \times 80$. These feature tensors are saved offline together with their class labels and language names, and then organized into train, validation, and test splits for each regional experiment. During training, the notebooks load these serialized tensors directly from disk rather than extracting features on the fly, which ensures that all architectures are trained from exactly the same input representation.

This storage strategy also standardizes data handling across the project. The preprocessed data are loaded into PyTorch Dataset and DataLoader objects, with separate loaders for the train, validation, and test splits. Because utterances have different durations, models that operate on full sequences use padded batches together with length information so that padded frames can be masked out during computation. Models that require fixed-length inputs instead apply a deterministic aggregation step to the same precomputed sequence representation. In this way, the project-wide workflow keeps preprocessing constant while allowing each architecture to define its own method of temporal aggregation.

3.2 Feed-Forward Neural Network (FNN)

For the FNN branch, all models use the shared precomputed acoustic feature sequences $\mathbf{X} \in \mathbb{R}^{T \times 80}$, where T is the number of frames and 80 is the feature dimension at each frame. The three FNN variants differ in how they aggregate temporal information before classification. For the first two variants, each utterance is reduced to a fixed-length representation by mean pooling over time:

$$\bar{\mathbf{x}} = \frac{1}{T} \sum_{t=1}^T \mathbf{x}_t, \quad \bar{\mathbf{x}} \in \mathbb{R}^{80} \quad (1)$$

where $\mathbf{x}_t$ denotes the 80-dimensional feature vector at frame t . This pooled vector is then passed into a multilayer perceptron. For the third variant, the full variable-length sequence is retained, padded within the batch, and paired with the true sequence lengths so that padding does not affect the learned attention weights.

FNN-1: Baseline Mean-Pooled FNN. The first model is a simple feed-forward neural network operating on the mean-pooled utterance vector. It contains three fully connected hidden layers with dimensions 256, 128, and 64, each followed by a ReLU activation and dropout with probability 0.3. The output layer maps the final hidden representation to the C language classes in the regional

subset. Its computation can be written as:

$$\mathbf{h}^{(1)} = \text{Dropout}(\text{ReLU}(\text{Linear}_{80 \rightarrow 256}(\tilde{\mathbf{x}}))) \quad (2)$$

$$\mathbf{h}^{(2)} = \text{Dropout}(\text{ReLU}(\text{Linear}_{256 \rightarrow 128}(\mathbf{h}^{(1)}))) \quad (3)$$

$$\mathbf{h}^{(3)} = \text{Dropout}(\text{ReLU}(\text{Linear}_{128 \rightarrow 64}(\mathbf{h}^{(2)}))) \quad (4)$$

$$\hat{\mathbf{y}} = \text{Linear}_{64 \rightarrow C}(\mathbf{h}^{(3)}) \quad (5)$$

This model serves as the baseline FNN and tests how much discriminative information remains after uniform temporal averaging.

FNN-2: Upgraded Mean-Pooled FNN. The second model uses the same 80-dimensional mean-pooled input, but increases capacity and adds batch normalization for more stable optimization. Its hidden layers have dimensions 512, 256, and 128, and each hidden transformation is followed by batch normalization, ReLU, and dropout with probability 0.35. The hidden-layer computation follows the pattern:

$$\mathbf{h}^{(l)} = \text{Dropout}(\text{ReLU}(\text{BN}(\text{Linear}(\mathbf{h}^{(l-1)})))) \quad (6)$$

with layer sizes $80 \rightarrow 512 \rightarrow 256 \rightarrow 128$, followed by a final output projection $\text{Linear}_{128 \rightarrow C}$. Compared with FNN-1, this model is wider, better regularized, and generally more stable during training.

FNN-3: FNN with Attention Pooling. The third model preserves the full sequence and learns which frames are most informative for language identification. Each input frame is first passed through a frame-level feed-forward encoder consisting of two linear layers with GELU activations and dropout:

$$\mathbf{r}_t = \text{Dropout}(\text{GELU}(\text{Linear}_{80 \rightarrow 256}(\mathbf{x}_t))) \quad (7)$$

$$\mathbf{h}_t = \text{Dropout}(\text{GELU}(\text{Linear}_{256 \rightarrow 256}(\mathbf{r}_t))) \quad (8)$$

This produces an encoded sequence $\{\mathbf{h}_1, \dots, \mathbf{h}_T\}$, where each $\mathbf{h}_t \in \mathbb{R}^{256}$. A learned attention layer then assigns a scalar score to each valid frame:

$$s_t = \text{Linear}_{256 \rightarrow 1}(\mathbf{h}_t) \quad (9)$$

After masking padded positions, a softmax over time converts these scores into normalized attention weights:

$$\alpha_t = \frac{\exp(s_t)}{\sum_{j=1}^T \exp(s_j)} \quad (10)$$

The utterance representation is then computed as a weighted sum of the encoded frames:

$$\mathbf{u} = \sum_{t=1}^T \alpha_t \mathbf{h}_t \quad (11)$$

Finally, the pooled representation is passed through a classifier head:

$$\mathbf{g} = \text{Dropout}(\text{ReLU}(\text{Linear}_{256 \rightarrow 256}(\mathbf{u}))) \quad (12)$$

$$\hat{\mathbf{y}} = \text{Linear}_{256 \rightarrow C}(\mathbf{g}) \quad (13)$$

Unlike the first two variants, this model does not collapse the sequence before learning. Instead, it preserves temporal information until the pooling stage and allows the network to emphasize acoustically informative regions of the utterance.

Training Setup. All three FNN variants are trained as multiclass classifiers using cross-entropy loss. Let $\hat{\mathbf{y}}$ denote the output logits

and $\hat{\mathbf{p}} = \text{softmax}(\hat{\mathbf{y}})$ the predicted class probabilities. The training objective is:

$$\mathcal{L}_{\text{CE}} = - \sum_{c=1}^C y_c \log \hat{p}_c \quad (14)$$

where y_c is the one-hot target for class c . The baseline model is optimized with Adam using a learning rate of 10^{-3} , while the stronger FNN variants are typically trained with AdamW using a learning rate of 3×10^{-4} and weight decay of 10^{-2} . During training, model performance is monitored on the validation set after each epoch, and the best-performing checkpoint is restored based on validation accuracy. In later attention-based experiments, additional stabilization techniques such as early stopping, gradient clipping, and label smoothing were also used in some regional notebooks. Final evaluation is carried out on the held-out test set using overall accuracy, confusion matrices, per-language accuracy, and full classification reports.

Among the three FNN variants, the attention-pooling model was chosen as the final FNN baseline for comparison with the other model families, including CNN-, LSTM-, and transformer-based models. Although it was not the top-performing variant in every regional subset, it was the strongest or most competitive FNN overall and provided a more informative temporal aggregation mechanism than the two mean-pooled baselines. Consequently, we use the FNN with attention pooling as the main feed-forward reference model in the final cross-model evaluation.

3.3 Convolutional Neural Network (CNN)

The CNN architecture is designed to operate directly on the variable-length log-mel spectrogram sequences produced by the shared preprocessing pipeline. Each utterance is represented as a tensor of shape $T \times 80$, where T denotes the number of acoustic frames and 80 is the number of mel-frequency bins. Prior to processing, the input is transposed to shape $B \times 80 \times T$, where B is the batch size, treating the mel-frequency dimension as the channel axis and the temporal dimension as the spatial axis over which convolution is applied.

Input Representation. Each raw waveform is converted into an 80-dimensional log-mel spectrogram using a short-time Fourier transform with an FFT size of 400 points and a hop length of 160 samples at a target sampling rate of 16 kHz. The resulting magnitude spectrogram is passed through a mel filterbank with 80 bins and subsequently transformed to a decibel scale via an amplitude-to-dB conversion. Per-utterance normalization is then applied by subtracting the global mean and dividing by the standard deviation of the log-mel values, followed by an additional per-frame normalization within the collation function to standardize each frequency bin across time. Variable-length sequences within a batch are zero-padded to the maximum utterance length, and the true frame counts are retained as length tensors for downstream compatibility.

Convolutional Blocks. The feature extraction backbone consists of three sequential one-dimensional convolutional blocks applied along the time axis. Given an input feature map $\mathbf{X} \in \mathbb{R}^{B \times F \times T}$, where B is the batch size, $F = 80$ is the mel-frequency dimension, and T is the number of time steps, each convolutional block applies

a 1D convolution, batch normalization, and a ReLU nonlinearity:

$$\mathbf{H}^{(l)} = \text{ReLU}\left(\text{BN}\left(\text{Conv1d}\left(\mathbf{H}^{(l-1)}\right)\right)\right) \quad (15)$$

The first block uses a kernel size of 5 with 128 output channels and symmetric padding of 2, followed by max-pooling with stride 2, which halves the temporal resolution. The second block doubles the channel depth to 256 with the same kernel size and padding, again followed by max-pooling with stride 2. The third block applies a kernel of size 3 with padding 1 and maintains 256 channels without additional pooling. In layer-by-layer form, the convolutional backbone is:

$$\begin{aligned} \text{Conv1d}(80, 128, 5) &\rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2) \\ \text{Conv1d}(128, 256, 5) &\rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2) \\ \text{Conv1d}(256, 256, 3) &\rightarrow \text{BN} \rightarrow \text{ReLU} \end{aligned}$$

This design progressively expands the receptive field while increasing representational capacity, enabling the network to capture short-term spectral patterns in early layers and broader temporal context in deeper layers.

Pooling Strategy. Following the final convolutional block, global average pooling is applied over the remaining time dimension, collapsing the output of shape $B \times 256 \times T'$ to a single fixed-length 256-dimensional utterance embedding per example:

$$\mathbf{h} = \frac{1}{T'} \sum_{t=1}^{T'} \mathbf{H}_{:,t}^{(3)} \quad (16)$$

This aggregation strategy produces a representation that is invariant to input duration and provides an efficient alternative to recurrent temporal aggregation for audio classification.

Classification Head. The pooled utterance embedding $\mathbf{h} \in \mathbb{R}^{256}$ is passed through a two-layer fully connected classifier head. The first linear layer projects the representation to a 128-dimensional hidden space, followed by a ReLU activation and dropout with probability 0.3 for regularization. The second linear layer maps the hidden representation to the output logits:

$$\hat{y} = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{h}) \quad (17)$$

where $\hat{y} \in \mathbb{R}^C$ and C is the number of languages in the regional subset. In layer-by-layer form, the classifier head is:

$$\text{Linear}(256, 128) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow \text{Linear}(128, C)$$

The CNN architecture is identical across all seven regional subsets; the sole region-specific parameter is the output dimension C of the final linear layer, which is set to match the number of languages in each regional group (e.g., $C = 14$ for South Asian, $C = 20$ for Sub-Saharan African).

Training Setup. The CNN is trained as a multi-class classifier using cross-entropy loss. Parameter optimization is performed using the Adam optimizer with a learning rate of 2×10^{-5} . Training is conducted for five epochs with a batch size of 64. Model performance is monitored on the validation set after each epoch, with training and validation loss and accuracy recorded throughout. Final evaluation is carried out on the held-out test set using overall accuracy, confusion matrices, and per-language accuracy, providing

a consistent basis for cross-model comparison. The same architecture and training configuration is applied independently across all seven regional language subsets examined in this project.

3.4 Long Short-Term Memory Network (LSTM)

The LSTM model is a type of recurrent neural network designed to learn long-term dependencies in sequential data [3]. Unlike vanilla RNNs, which struggle to propagate gradients across long sequences due to the vanishing gradient problem, LSTMs introduce a cell state c_t that acts as a long-term memory channel running alongside the hidden state h_t . Changes to the cell state are controlled by a gating mechanism, which allows the LSTM to selectively remember and forget information over arbitrarily long input sequences. This makes LSTMs well suited to speech tasks where relevant patterns may be separated by many frames.

Gating Mechanism. The cell state is modified by three learned gates. The forget gate determines how much of the previous cell state to retain, the input gate controls how much new information from the current timestep is written into the cell state, and the output gate determines what portion of the cell state is exposed as the hidden state h_t . Together these gates give the LSTM fine-grained control over what information is preserved or discarded at each step of the sequence.

Bidirectional Architecture. We utilize a two-layer bidirectional LSTM [6], which processes each sequence in both the forward and backward directions independently. At each timestep, the hidden states from both directions are concatenated. This allows the model to capture dependencies from both past and future context, which is particularly useful for language identification since phonological patterns are better characterized with access to full-utterance context rather than only the frames preceding a given timestep.

Classification Head. The final utterance-level representation is formed by concatenating the last hidden states of the forward and backward passes from the final LSTM layer, producing a fixed-size vector that summarizes the entire sequence. This vector is passed through a classification head consisting of a LayerNorm, a dropout layer, and a linear projection to the number of target classes. LayerNorm is applied before the linear layer to stabilize the distribution of hidden state activations, which can vary significantly across utterances of different lengths and speakers.

Training Setup. The LSTM was configured with a hidden dimension of 128 per direction, giving a total hidden state size of 256 after concatenating the forward and backward final hidden states. Dropout with a probability of 0.3 was applied within the classification head to regularize the model and reduce overfitting to speaker-specific features in the training set. The model was trained using the Adam optimizer with a learning rate of $3e-4$ and weight decay of $1e-3$, with the learning rate annealed to zero over the course of training using a cosine annealing schedule. Cross-entropy loss was used as the training objective with label smoothing of 0.1, which penalizes overconfident predictions by redistributing a small portion of probability mass from the correct class to all other classes, encouraging more balanced class probabilities throughout optimization. Training was run for a maximum of 100 epochs with early stopping applied when validation accuracy failed to improve

for 20 consecutive epochs, with the best checkpoint selected based on validation accuracy and used for final test evaluation.

3.5 Transformer-Based Model

To model temporal dependencies in raw audio signals, we adopt the *wav2vec 2.0* architecture [1], a self-supervised speech representation model that combines convolutional feature extraction with a Transformer-based context network. In this work, we utilize the pretrained *wav2vec2-xls-r-300m* model, which has been trained on large-scale multilingual speech corpora including the LibriVox and Librispeech [5] datasets.

Feature Encoder. Given an input waveform ($a \in \mathbb{R}^{B \times L}$), where (B) is the batch size and (L) is the number of audio samples, a convolutional neural network (CNN) feature encoder is first applied to extract latent acoustic representations. The encoder consists of a stack of 7 1-D convolutional layers that progressively down-sample the raw signal and capture local temporal patterns such as phonetic transitions. The resulting feature sequence is:

$$H = \text{CNN}(a), \quad H \in \mathbb{R}^{B \times T \times C}$$

where (T) is the number of time steps after down-sampling and (C=768) is the number of feature channels.

Feature Projection. The CNN output is then projected into the Transformer embedding space via a linear transformation:

$$X = \text{Proj}(H), \quad X \in \mathbb{R}^{B \times T \times d}$$

where (d) is the model dimension (e.g., (d=1024) for XLS-R 300M). This step aligns the feature representation with the input requirements of the Transformer.

Convolutional Positional Embedding. Unlike standard Transformers that use fixed or learned absolute positional embeddings, *wav2vec 2.0* employs a tempoconvolutional positional embedding to encode temporal order. A 1D temporal convolution is applied over the feature sequence, followed by a nonlinearity and residual addition:

$$X' = \text{LayerNorm}(X + \text{GELU}(\text{Conv}_{\text{pos}}(X)))$$

$$(\text{Conv}_{\text{pos}}(X))_t = \sum_{i=-1}^1 W_i X_{t+i}$$

This formulation allows the model to capture relative positional information and local temporal structure, which is particularly suitable for continuous speech signals.

Transformer Context Network. The position-enhanced features (X') are then passed through a stack of (L) Transformer encoder layers [2]. Each layer consists of multi-head self-attention and a feed-forward network (FFN), with residual connections and layer normalization:

$$Z = X' + \text{MHA}(\text{LayerNorm}(X'))$$

$$Y = Z + \text{FFN}(\text{LayerNorm}(Z))$$

The multi-head self-attention mechanism enables the model to capture long-range dependencies across the audio sequence, while the FFN introduces nonlinear transformations:

$$\text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 x)$$

For the XLS-R 300M configuration, the context network consists of L=12 Transformer layers with model dimension (d=1024), feed-forward dimension (4096), and 8 attention heads.

Classification Head. For downstream audio classification, the contextualized representations produced by the Transformer are aggregated (e.g., via mean pooling or max pooling) and passed to a language ID classification head.

$$\hat{y} = \text{Linear}(h)$$

where (h) is the pooled representation and ($\hat{y}$) are the predicted class probabilities.

Summary. Overall, the *wav2vec 2.0* architecture effectively combines local feature extraction through convolution with global context modeling via self-attention. This hybrid design is well-suited for speech processing tasks, as it captures both short-term acoustic patterns and long-range temporal dependencies necessary for robust audio classification.

3.6 Training Procedure and Evaluation Metrics

All models in this project were trained using the same data pipeline to ensure fair comparison across architectures. The audio samples were first converted into log-mel spectrograms, which served as input features for the models. Due to the variable length of the audio recordings, the sequences were padded within each batch, and a custom collate function was used to efficiently prepare the batched input.

To address issues with speaker-specific overfitting present in the original dataset splits, we took all of the data in the training split and manually divided it into new training, validation, and test sets. This ensured a more balanced distribution of speakers across the splits and improved the model's ability to generalize to unseen data. This reshuffling was done only for the FFN and CNN models, not the LSTM and transformer-based models.

For evaluation, we used classification accuracy as the primary metric for both the validation and test sets. In addition, training accuracy was tracked to compare against validation performance and identify overfitting. The final model comparisons were based on training loss and test accuracy, allowing us to assess how well each model distinguished between closely related languages.

4 Results

The following table shows the empirical performance of the different model families across the regional language subsets.

Table 1: Test accuracy (%) across the seven regional subsets for all model families.

Regional Subset	FNN+Attn	CNN	LSTM	Transformer
Western European	14.00	27.92	10.35	91.67
Eastern European	17.57	60.75	10.87	94.59
South Asian	22.29	50.86	19.11	87.89
CJK	40.31	92.25	33.90	–
South-East Asian	20.05	60.91	29.83	87.32
Central Asia + MENA	22.46	76.50	23.35	94.41
Sub-Saharan African	21.46	31.35	14.39	–

**Note: could not train transformer on CJK and Sub-Saharan African regions due to limited computing power.*

4.1 Overall Model Comparison

Across all regional subsets, the Transformer and CNN models consistently outperformed both the FNN+Attn and LSTM models, often by a significant margin. This suggests that Transformer architectures have the ability to model long-range dependencies and capture global contextual relationships within the audio sequence. Additionally, convolutional architectures are particularly effective at capturing local temporal and spectral patterns in log-mel spectrograms, which are critical for distinguishing between acoustically similar languages.

The LSTM model generally performed the worst, indicating that sequential modeling alone may not be sufficient to capture the fine-grained phonetic distinctions required for this task. The FNN+Attn model showed moderate performance but lacked the ability to effectively model localized acoustic structures compared to the CNN and transformer.

4.2 Region-Specific Results

The results in Table 1 highlight clear differences in model performance across geographic regions. Certain regions are inherently easier for language classification, while others present greater challenges due to increased linguistic similarity and a larger number of classes.

Among all regions, the CJK (Chinese, Japanese, Korean) subset achieved the highest accuracies across models. This is likely due to the relatively small number of languages (only 4) in the subset, as well as the more distinct phonetic and acoustic characteristics between these languages. In contrast, the Western European subset consistently exhibited the lowest performance. This can be attributed to the significantly larger number of languages in the group (approximately 26), as well as the high degree of phonetic similarity between many of these languages, which reduces class separability.

Regions such as Eastern European and South-East Asian were consistently average; these subsets contain a comparable number of languages, which likely contributes to their similar accuracy levels. However, their performance remains lower than CJK, suggesting that both the number of classes and the degree of phonetic overlap influence classification difficulty.

The Central Asia + MENA, South Asian, and Sub-Saharan African regions also showed varied performance, indicating that regional linguistic diversity and dataset composition play an important role in model effectiveness.

Across all regions, the Transformer-based model significantly outperformed the other architectures, demonstrating its strength in capturing both local and global patterns in speech data. The CNN consistently achieved the second-best performance, suggesting that convolutional models are well-suited for extracting meaning from acoustic features. In contrast, the LSTM and FNN+Attn models underperformed, indicating limitations in their ability to distinguish between closely related languages in this setting.

5 Conclusion and Discussion

In this project, we explored the task of identifying languages from audio-based data across regions using multiple neural network architectures. Our results demonstrate that distinguishing between closely related languages is significantly more challenging than

general multilingual classification, particularly in regions with a plethora of languages and high phonetic similarity.

Among the models evaluated, the Transformer model achieved the highest performance in all regions, and CNNs also showed strong results. Additionally, we found that dataset structure, particularly speaker distribution and class composition, plays a critical role in model performance and generalization.

Future work could focus on improving the robustness and generalization of the model through enhanced data preprocessing and model design. For example, applying more advanced audio normalization and noise reduction techniques could help reduce variability caused by differences in recording conditions and speakers. Additionally, exploring more advanced architectures or pre-trained speech models could improve performance in some regions. Finally, expanding the dataset or balancing class distributions across regions may help address performance disparities observed between different language groups.

Overall, this work highlights the importance of both model selection and data design when addressing fine-grained language classification tasks, and suggests that more advanced architectures are better suited for capturing subtle acoustic differences in speech.

References

- [1] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *CoRR*, abs/2006.11477, 2020.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [4] Hugging Face. google/fleurs: Dataset loading script, 2026. Accessed: 2026-02-03.
- [5] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: An asr corpus based on public domain audio books. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5206–5210, 2015.
- [6] Mike Schuster and Kuldip K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997.

A Team Member Contributions

A.1 Keerthana Girish

Keerthana worked on creating, training, and testing the CNN architecture with Medha. She conducted experiments with hyperparameter tuning, including variations in batch size, learning rate, and model configuration, and explored feature engineering techniques to improve classification performance across all regions. She presented the introduction/motivation and CNN results slides during the final presentation.

A.2 Medha Bhattacharya

Medha worked on creating, training, and testing the CNN architecture with Keerthana. She assisted in designing the convolutional backbone and classification head, and contributed to the preprocessing and data loading pipelines used across regional subsets. She helped evaluate CNN performance through confusion matrix analysis and per-language accuracy breakdowns, and contributed to writing the CNN section of the report. She also presented the CNN architecture and workflow setup slides during the final presentation.

A.3 Melina Pouya

Melina worked on the feed-forward neural network (FNN) branch of the project and helped organize the shared data pipeline used across experiments. She created, trained, and evaluated three FNN variants

across the regional language subsets, including the final attention-pooling FNN used for comparison with the other architectures. She also contributed to the preprocessing and evaluation setup, including confusion-matrix and test-accuracy analysis, and helped write the Data, preprocessing, FNN, and Results sections of the report.

A.4 Vincent Wang

Vincent primarily worked on creating and training the transformer model for every region. He initially found the model through research, created the training pipeline and to finetune the transformer for the fleurs dataset. He also experimented with different training setups, including using LoRA to finetune the model backbone and using data augmentation such as random noise and random pitch perturbing to improve accuracy. He also presented the transformer model slides and results in the final presentation.

A.5 Eric Cao

Eric primarily worked on creating and training the LSTM model. This included playing with various hyperparameters to improve the model's performance and performing several data augmentation methods to reduce model overfitting. Accordingly, Eric created and presented the LSTM slides during the Project Presentation, and wrote the LSTM section in this paper. He also briefly tried (and failed) to train an LSTM model on all 102 languages in FLEURS.

Received March 2026